Выполнил: Василенко И.А Группа: ПИа-090303-о24

ЛАБОРАТОРНАЯ РАБОТА №9

Дисциплина: Основы программирования на Python

Тема: Сетевое взаимодействие в Python: TCP/UDP, клиент-серверная архитектура и прикладные протоколы

Вариант 2: Калькулятор

Цель работы

Получить практические навыки разработки сетевых приложений на Python и понять принципы клиент-серверного взаимодействия. В ходе работы необходимо изучить базовые принципы сетевого взаимодействия, IP-адреса и порты, понятия TCP и UDP, освоить модуль socket, реализовать TCP-сервер и TCP-клиент, разработать собственный простой протокол прикладного уровня, организовать передачу структурированных данных в формате JSON, реализовать обработку нескольких клиентов, научиться обрабатывать разрывы соединения, тайм-ауты и ошибки, отделить сетевой уровень от бизнес-логики и написать автоматические тесты для протокола и бизнес-логики.

Постановка индивидуального задания (Вариант 2. Калькулятор)

Реализовать сетевой калькулятор. Сущность Calculation содержит поля id, expression, result и timestamp. Специализированная команда calculate принимает выражение, например {"command": "calculate", "data": {"expression": "15 * 4 + 2"}}, сервер вычисляет результат, сохраняет операцию в историю и возвращает её клиенту. Помимо расчёта сервер поддерживает просмотр истории, получение записи по идентификатору, изменение, удаление, очистку истории и проверку доступности.

Краткие теоретические сведения

Клиент-серверная архитектура предполагает разделение ролей: сервер ожидает подключения на известном адресе и порту и обрабатывает запросы, а клиент инициирует соединение и отправляет запросы. IP-адрес определяет узел в сети, порт — конкретное приложение на этом узле. В лабораторной работе используется адрес 127.0.0.1 (петлевой интерфейс) и порт 5001.

Протокол TCP обеспечивает установление соединения, гарантированную доставку и сохранение порядка байтов, поэтому применяется в задачах, где важна целостность данных. Протокол UDP не устанавливает соединение и не гарантирует доставку, но имеет меньшие накладные расходы. Для обмена текстовыми командами выбран TCP.

Сокет — программный интерфейс для обмена данными по сети. В Python он создаётся функцией socket.socket(socket.AF_INET, socket.SOCK_STREAM), после чего сервер выполняет bind(), listen() и accept(), а клиент — connect(). Поскольку TCP является потоком байтов, границы сообщений необходимо задавать самостоятельно; в работе для этого используется разделитель-перевод строки.

Прикладной протокол определяет формат команд и ответов. В качестве формата выбран JSON: он компактно представляет вложенные структуры, удобен для чтения и поддерживается стандартным модулем json. Каждое сообщение передаётся одной строкой в кодировке UTF-8 и завершается символом перевода строки.

Архитектура приложения

Приложение разделено на уровни, каждый из которых отвечает за свою часть работы. Сетевой уровень не содержит предметной логики, а бизнес-логика ничего не знает о сокетах.

Компонент	Назначение
client.py	Класс CalculatorClient: подключение к серверу, формирование запросов, вывод ответов, интерактивный режим

Компонент	Назначение
server.py	Класс CalculatorServer: приём подключений, чтение потока байтов, разбор команд, формирование ответов
protocol.py	Кодирование и декодирование сообщений и ответов, проверка обязательных полей
service.py	Класс CalculatorService: валидация выражения, вычисление, работа с историей
repository.py	Класс CalculationRepository: хранение вычислений в памяти, генерация идентификаторов
models.py, exceptions.py	Сущность Calculation и иерархия исключений ProtocolError, CalculationError, CalculationNotFoundError

Цепочка обработки запроса выглядит следующим образом: клиент формирует словарь с командой и данными, протокол кодирует его в строку JSON, сервер принимает байты, выделяет сообщения по разделителю и передаёт разобранный словарь диспетчеру команд, который вызывает соответствующий метод сервиса, а сервис обращается к репозиторию. Ответ проходит обратный путь и возвращается клиенту.

Разработанный протокол

Запрос клиента содержит обязательное поле command, необязательное поле data с параметрами и необязательное поле request_id, которое возвращается в ответе и позволяет сопоставить запрос с ответом. Ответ содержит поле status со значением ok или error, поле data с результатом либо поле error с текстом ошибки.

Команда	Запрос	Ответ	Назначение
calculate	data.expression	data: запись Calculation	Вычислить выражение и сохранить в историю
list	data отсутствует	data: массив записей	Получить все вычисления
get	data.id	data: запись Calculation	Получить вычисление по идентификатору
update	data.id, data.expression	data: запись Calculation	Пересчитать и изменить запись
delete	data.id	data.deleted	Удалить запись из истории
history	data.limit	data: массив записей	Получить последние вычисления
clear	data отсутствует	data.cleared	Очистить историю

Команда	Запрос	Ответ	Назначение
ping	data отсутствует	data.message = pong	Проверить доступность сервера

При нарушении формата сообщения возбуждается ProtocolError, и сервер возвращает ответ со статусом error, не прерывая работу с клиентом. Неизвестная команда также приводит к ответу с ошибкой.

Сервер и обработка нескольких клиентов

Сервер создаёт TCP-сокет, включает опцию SO_REUSEADDR, связывает его с адресом и портом и переводит в режим прослушивания. Основной цикл принимает подключения и передаёт каждого клиента в пул потоков ThreadPoolExecutor с десятью рабочими потоками, поэтому несколько клиентов обслуживаются одновременно и медленный клиент не блокирует остальных.

Каждому соединению устанавливается тайм-аут 30 секунд, что позволяет освобождать ресурсы при зависшем клиенте. Данные читаются порциями по 4096 байт и накапливаются в буфере; как только в буфере появляется символ перевода строки, из него извлекается очередное сообщение и обрабатывается. При разрыве соединения и при ошибке чтения предусмотрены отдельные обработчики, а блок finally гарантирует закрытие сокета клиента. Обработка исключений разделена по типам: отсутствие записи и ошибка вычисления возвращаются как ошибки предметной логики, некорректные параметры — как ошибка формата, а непредвиденные ситуации журналируются через logging.

Листинг 9.1 — Чтение потока и диспетчеризация команд (server.py)

```
def _client_thread(self, conn: socket.socket, addr) -> None:
    conn.settimeout(CLIENT_TIMEOUT)
    buffer = b""
    try:
        while True:
            try:
                chunk = conn.recv(BUFFER_SIZE)
            except socket.timeout:
                logger.warning("Client %s timeout", addr)
                break
            if not chunk:
                break
            buffer += chunk
            while b"
" in buffer:
                line, buffer = buffer.split(b"
```

```
", 1)
                if not line.strip():
                    continue
                try:
                    message = decode_message(line)
                    response = self._handle_command(message)
                except ProtocolError as e:
                    response = {"status": "error", "error": str(e)}
                conn.sendall(encode_response(**response))
    except ConnectionResetError:
        logger.warning("Client %s reset connection", addr)
    finally:
        conn.close()
```

Бизнес-логика и безопасность вычислений

Вычисление выражения выполняется функцией eval с пустым словарём встроенных функций, что исключает доступ к функциям и модулям языка. Перед вычислением выражение проходит валидацию: оно не должно быть пустым, не должно содержать символов вне набора цифр, пробелов, операторов и скобок, а также не должно содержать двух операторов подряд. Символ ^ заменяется на оператор возведения в степень **.

Отдельно обрабатывается деление на ноль: исключение ZeroDivisionError преобразуется в CalculationError с понятным сообщением. После вычисления проверяется тип результата: логические значения и нечисловые результаты отклоняются. Все успешные вычисления сохраняются в репозитории с уникальным идентификатором и отметкой времени, а метод history() возвращает записи в обратном порядке с ограничением количества.

Листинг 9.2 — Валидация и вычисление выражения (service.py)

```
_SAFE_PATTERN = re.compile(r"^[ds+-*/().,%^]+$")

def _validate_expression(self, expression: str) -> None:
    if not expression or not expression.strip():
        raise CalculationError("Выражение пустое")
    if not _SAFE_PATTERN.match(expression):
        raise CalculationError("Выражение содержит недопустимые символы")
    normalized = expression.replace("**", "*")
    if re.search(r"[+-*/^%]{2,}", normalized):
        raise CalculationError("Недопустимая последовательность операторов")


def _evaluate(self, expression: str) -> float:
    expr = expression.replace("^", "**")
    try:
        result = eval(expr, {"__builtins__": {}}, {})
    except ZeroDivisionError:
        raise CalculationError("Деление на ноль")
    except Exception as e:
        raise CalculationError(f"Некорректное выражение: {e}")
```

```
    if isinstance(result, bool) or not isinstance(result, (int, float)):
        raise CalculationError("Выражение не даёт числовой результат")
    return float(result)
```

Раздел тестирования

Автоматические тесты разделены по уровням. Тесты протокола проверяют кодирование и декодирование сообщений и ответов, тесты репозитория — операции хранения, тесты сервиса — вычисления и обработку ошибок, интеграционные тесты поднимают реальный сервер на свободном порту и обращаются к нему по TCP. Всего выполнено тридцать шесть тестов.

Файл тестов	Кол-во	Что проверяется
tests/test_protocol.py	7	Кодирование и декодирование JSON, отсутствие request_id, некорректный JSON, отсутствие command, поле data не является объектом, ответы ok и error
tests/test_repository.py	8	Добавление и получение записи, уникальность идентификаторов, получение списка, изменение существующей и отсутствующей записи, удаление, очистка и сброс счётчика
tests/test_service.py	16	Приоритет операций, степень, деление, скобки, отрицательные числа, пустое выражение, деление на ноль, недопустимые символы, два оператора подряд, отсутствующая запись, изменение, история и её порядок, некорректный limit, очистка
tests/test_integration.py	5	Сквозной расчёт через TCP, команда list, неизвестная команда, ping, удаление отсутствующей записи

Интеграционные тесты самостоятельно находят свободный порт, запускают сервер в отдельном потоке, отправляют запросы через сокет и проверяют полученные ответы, что подтверждает работоспособность протокола в реальном сетевом взаимодействии. Дополнительно работа проверялась вручную: сервер запускался командой python server.py, клиент — командой python client.py; выполнялись вычисления 15 * 4 + 2, 2 ** 10 и (10 + 5) / 3, просматривалась история, проверялась реакция на деление на ноль и на выражение с буквами.

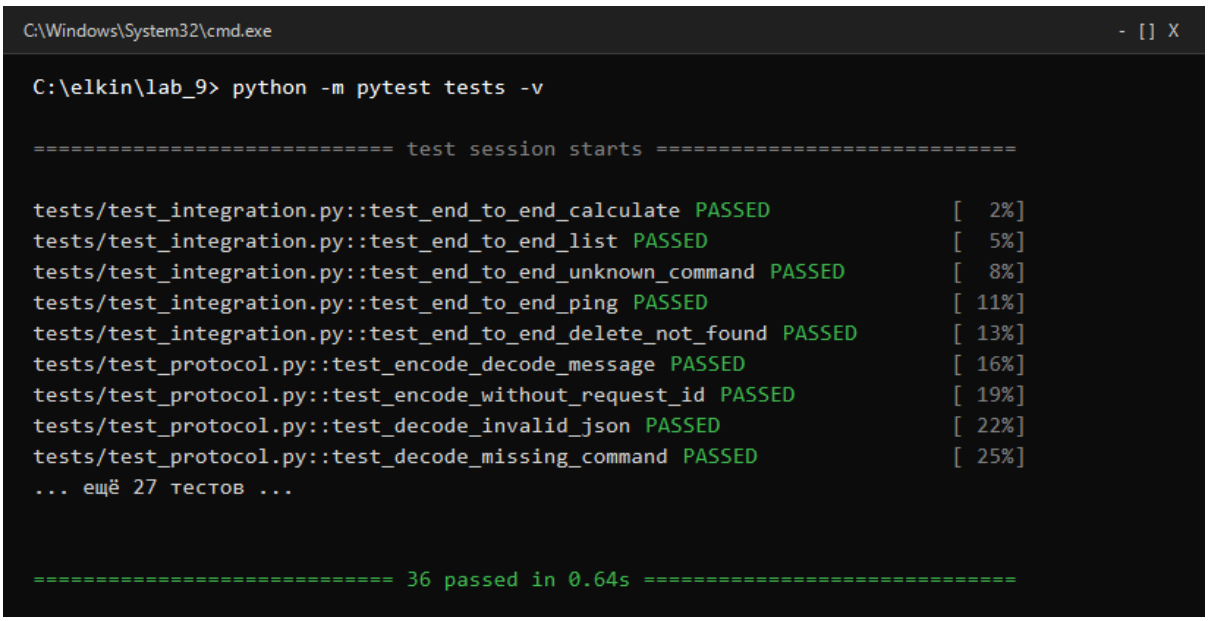


Рисунок 9.1 — Результат прохождения тестов протокола, репозитория, сервиса и интеграционных тестов

Листинги ключевых фрагментов программы

Листинг 9.3 — Кодирование и проверка сообщений (protocol.py)

```
def encode_message(command: str, data: dict | None = None,
                   request_id: str | None = None) -> bytes:
    message: dict = {"command": command, "data": data or {}}
    if request_id is not None:
        message["request_id"] = request_id
    return (json.dumps(message, ensure_ascii=False) + "
").encode("utf-8")


def decode_message(raw: bytes) -> dict:
    try:
        text = raw.decode("utf-8").strip()
        if not text:
            raise ProtocolError("Пустое сообщение")
        message = json.loads(text)
    except (UnicodeDecodeError, json.JSONDecodeError) as e:
        raise ProtocolError(f"Некорректный JSON: {e}")


    if not isinstance(message, dict):
        raise ProtocolError("Сообщение должно быть JSON-объектом")
    if "command" not in message or not isinstance(message["command"], str):
        raise ProtocolError("Отсутствует обязательное поле 'command'")
    if "data" in message and not isinstance(message["data"], dict):
        raise ProtocolError("Поле 'data' должно быть объектом")
    return message
```

Листинг 9.4 — Запуск сервера с пулом потоков (server.py)

```
def start(self) -> None:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server.bind((self.host, self.port))
        server.listen(5)
        logger.info("Server started on %s:%d", self.host, self.port)
```

```
try:
    while True:
        conn, addr = server.accept()
        self._executor.submit(self._client_thread, conn, addr)
except KeyboardInterrupt:
    logger.info("Server stopped by user")
finally:
    self._executor.shutdown(wait=False)
```

Вывод

В ходе выполнения лабораторной работы разработано клиент-серверное приложение калькулятора, работающее по протоколу TCP. Освоены создание сокетов, обработка нескольких клиентов через пул потоков, чтение потока байтов с выделением сообщений по разделителю и обработка тайм-аутов и разрывов соединения. Разработан собственный прикладной протокол на основе JSON с командами calculate, list, get, update, delete, history, clear и ping. Разделение на сетевой, сервисный и складской уровни позволило тестировать бизнес-логику независимо от сокетов: тридцать шесть автоматических тестов проходят успешно.